Day => 3

ArrayList => It is resizable array implementation in java,
It is part of java collection framework

Properties => (1) dynamic in size
(2) store only objects

Concept of autoboxing → Concept of unboxing

<Integer> ← object

10, 20 → primitive type → convert into Object → execution time → convert into primitive types

So from primitive to wrapper class → autoboxing
from wrapper classes to primitive → unboxing

We need wrapper classes

(3) Ordered collection → stay in order what we added

(4) Add duplicates and null values

Types of implementing ArrayList

(1) Without generic

```
ArrayList list = new ArrayList();
list.add(1);
list.add("2");
int a = (int) list.get(0);
String b = (String) list.get(1);
```

typecast → for accessing 0 index

One case where need not to typecast :

```
System.out.println(list.get(0));
```

→ it will call internally toString() method which will convert everything to String.

b in this case need not to typecaste

(2) With generics

```
ArrayList<String> list = new ArrayList<>();
```

Constructor Summary

Constructors

Constructor	Description
`ArrayList()`	Constructs an empty list with an initial capacity of ten.
`ArrayList(int initialCapacity)`	Constructs an empty list with the specified initial capacity.
`ArrayList(Collection<? extends E> c)`	Constructs a list containing the elements of the specified collect they are returned by the collection's iterator.

Always use of generic ArrayList is more due to safety

Concept

Size of arrayList

old_capacity + (old_capacity >> 1)

Or

old_capacity + (old_capacity / 2)

→ if the old capacity is full then new arraylist is created with the above formula

and

elements are copied from old arraylist to new arraylist and old arraylist is removed by garbage collection

Concept of binding should be known.